

Coding & Solution

Date	30 June 2026
Team ID	
Project Name	OptiCrop: Smart Agricultural Production Optimization Engine
Maximum Marks	5 Marks

Coding & Solution

This section evaluates the quality and completeness of your implemented code and the overall solution. Your submission should demonstrate working functionality, clean code practices, and alignment with your defined requirements.

Instructions:

1. Ensure your code is well-structured, commented, and follows best practices for your chosen technology stack.
2. The solution should address the core problem statement and implement the key functional requirements.
3. Include all source files, configuration files, and setup instructions needed to run the application.
4. Attach or link to your code repository (e.g., GitHub) in the table below.

Solution Summary

Field	Details
Repository Link / URL	<i>Paste your GitHub repository URL here (e.g., https://github.com/your-username/OptiCrop)</i>
Programming Language(s)	Python, HTML5, CSS3, JavaScript
Framework(s) Used	Flask, Scikit-learn
Key Features Implemented	• Intelligent crop recommendation using Machine Learning. • Data preprocessing (missing values, outlier handling, feature scaling). • Comparison of multiple ML models (Logistic Regression, KNN, Decision Tree, Random Forest, K-Means). • Best model selection and serialization using Joblib. • Flask-based backend for prediction. • Interactive web interface for user input. • Input validation and error handling. • Real-time crop prediction based on soil and environmental parameters.
Pending / Incomplete Features	• Real-time weather API integration. • User authentication and profile management. • Cloud deployment (AWS/Render/Docker). • Mobile application support. • Fertilizer and irrigation recommendation module.

Setup / Run Instructions	1. Install Python 3.x and required libraries (pip install numpy pandas scikit-learn matplotlib seaborn flask joblib). 2. Place Crop_recommendation.csv inside the data folder. 3. Run python train.py to train the model and generate crop_model.joblib and scaler.joblib. 4. Run python app.py to start the Flask server. 5. Open http://localhost:5000 in a web browser. 6. Enter soil and environmental parameters to receive crop recommendations.
---------------------------------	---

Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions/classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments/documentation where necessary	Yes
4	Error handling is implemented for critical operations	Yes
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	Yes

Additional Notes / Comments:

The solution follows a modular architecture with separate components for data preprocessing, machine learning model training, backend API development, and frontend user interface.

- Multiple machine learning algorithms were evaluated, and the best-performing model (Random Forest) was selected based on classification performance.
- The application provides fast, accurate, and user-friendly crop recommendations using soil nutrients and environmental parameters.
- The project is designed for future enhancements, including real-time weather API integration, cloud deployment, and advanced smart farming features.